



算法设计与分析

Analysis and Design of Algorithm

任课教师：熊润群

办公室：计算机楼368室
Email: rxiong@seu.edu.cn



参考书目

主要教材

- 王晓东. 计算机算法设计与分析. 电子工业出版社

参考教材

1. Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman. **The Design and Analysis of Computer Algorithms**. 1974年影印本, 铁道出版社
2. Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman. **数据结构与算法**. 1983年影印本, 清华大学出版社
3. Thomas H. Cormen 等4人. **算法导论**. MIT第2版影印本, 高教出版社
4. 沈孝钧. **计算机算法基础**. 机械工业出版社



课程信息

■ 内容分配

- 课堂教学：01-08周，32学时
- 上机实验：09-16周，32学时

■ 考核方式

- 平时（点名、作业、上机）：**40%**
- 期末（考试）：**60%**

■ 作业要求

- 习题类型：算法分析题（一般3道题，LaTeX/MD）
- 截止日期：一周之内完成
- 提交方式：PDF电子版（学号-姓名-第x次作业.pdf）
- 发送邮件：(助教-陈铎文)

■ 答疑方式：计算机楼368，QQ群：xxxxxxx

什么是算法?
Algorithm

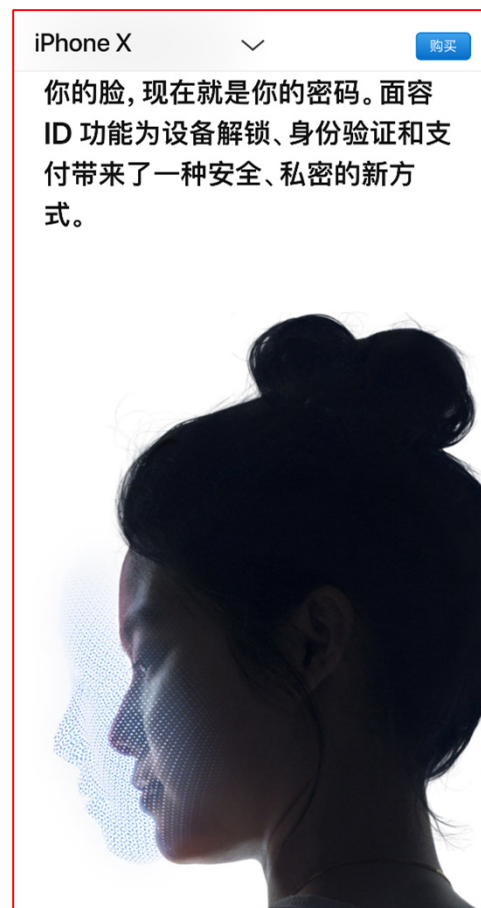
先来看几个生活中的例子



概率算法



寻路算法

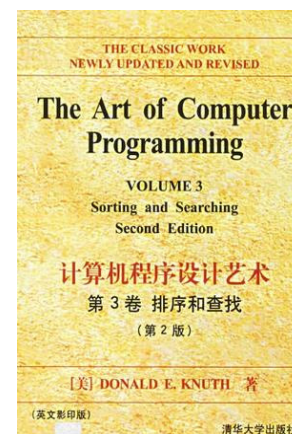
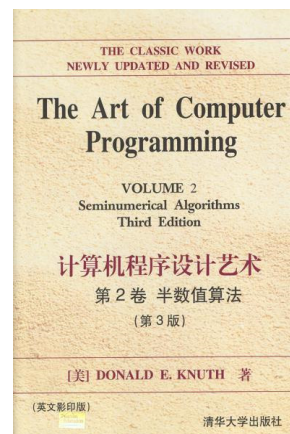
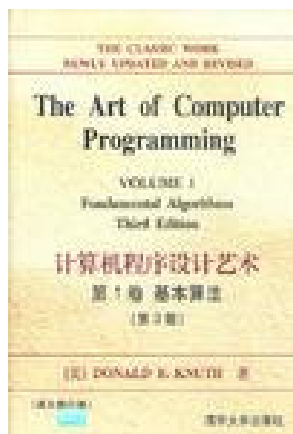


图像识别算法

什么是算法(Algorithm)

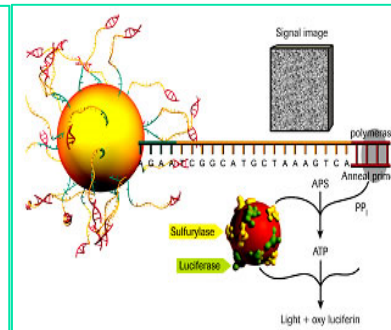
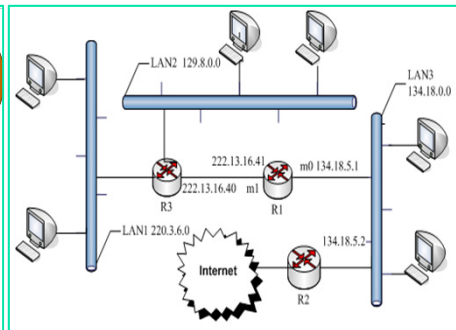
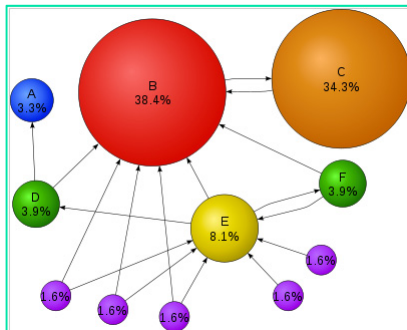
百度百科：算法是指解题方案的准确而完整的描述，是一系列解决问题的清晰指令，代表着用系统的方法描述解决问题的策略机制。

算法和程序设计技术的先驱者Donald Knuth：算法是带有输入输出的、有限的、确定的、有效的过程。



算法有哪些应用

- **互联网**：网页搜索、网络路由、BitTorrent...
- **生物信息**：人类基因组计划、蛋白质结构分析...
- **计算机图形**：电影、游戏、虚拟现实...
- **计算机安全**：手机、电子商务、投票系统...
- **多媒体**：MP3、JPG、HDTV...
- **人工智能**：人脸识别、AlphaGo、ChatGPT...
- **社会网络**：推荐系统、新闻推送、广告...
- **物理学**：粒子对撞机、反物质和暗物质探测...



本科生学算法有什么用

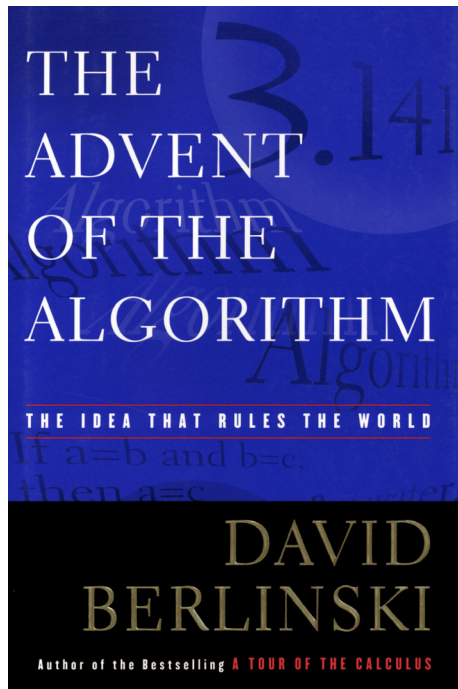
求职面试： GFM、BAT、TMD ...

读研深造： 考研面试、论文专利 ...

创新创业： 开发产品、性能优化 ...



科学殿堂里陈列着两颗熠熠生辉的宝石，一颗是**微积分**，另一颗就是**算法**。微积分成就了现代科学，而算法成就了现代世界。



David Berlinski
The Advent of the Algorithm
2000



本课程与其他课程的关系

编译
原理

操作
系统

计算机
网络

数据库
原理

算法设计与分析

数据结构

程序设计基础及语言

面向对象程序设计



本课程的内容

NP完全性理论与近似算法

高级理论

随机化算法

线性规划与网络流

高级算法

**递归
分治**

**动态
规划**

**贪心
算法**

**回溯与
分支限界**

基础算法

算法分析与问题的计算复杂性

基础理论

第一章 算法概述



第一章 算法概述

■ 学习要点:

- 理解算法的概念
- 理解什么是程序，程序与算法的区别和联系
- 掌握描述算法的方法
- 掌握算法的计算复杂性概念
- 掌握算法渐近复杂性的数学表述
- 了解NP类问题的基本概念

基础知识



算法的定义

- 算法(Algorithm)
 - 一个（由人或机器进行）关于某种运算规则的集合



确定性 清晰、无歧义

有限性 指令执行次数、时间

- 特点：
 - 执行时，不能包含任何主观的决定；
 - 不能有类似**直觉/创造力**等因素。

例子:

- 日常生活中做菜的过程，可否用算法描述？

- ✓ 如：“淡了”、“放点盐”、“再煮一会”。
- ✓ 可否用计算机完成？



- 算法必须规定明确的量与时间；
- 不能有含糊字眼。

算法描述：

```
if (咸度 < 5){ 放1克盐; 煮半分钟; }
```


随机算法与近似算法

- 当然不是所有算法都有明确的选择，有些按照一定的概率进行选择。

- “随机” 不等于 “随意”



- 有些问题没有实用算法（算精确解需要多年）。

✓ 去寻找{规则集}



近似算法

可以预测误差，
且误差足够小

启发式算法

误差无法控制，
但可预计误差大小

在可接受的时间内可以算出足够好的近似解



算法和程序的关系

- **程序**是算法用某种程序设计语言的具体实现
- **算法和程序**的区别主要在于：
 - 在**语言描述**上，程序必须是用规定的程序设计语言来写，而算法很随意；
 - 在**执行时间**上，算法所描述的步骤一定是有限的，而程序可以无限地执行下去。
- 例如：操作系统是程序，但不是算法

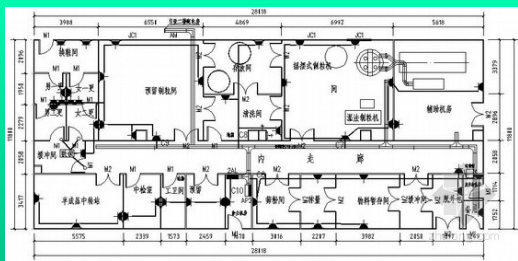


算法和数据结构的关系

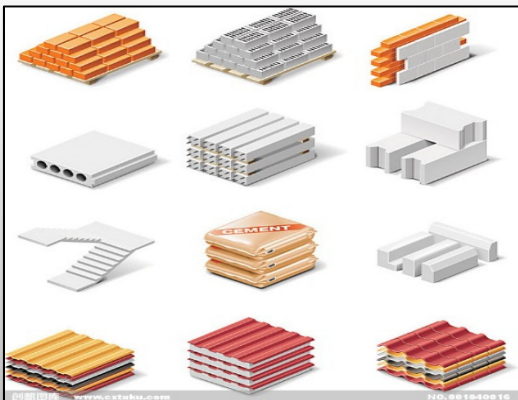
- 广义上讲，算法是一系列的运算步骤，它表达**解决某一类计算问题的一般方法**，对这类方法的任何一个输入，它可以按步骤一步一步计算，最终产生一个输出。
- 但是对于所有的计算问题，都**离不开要计算的對象或者要处理的信息**，而如何高效的把它们组织起来，就是数据结构关心的问题，所以算法是离不开数据结构的。

程序、算法和数据结构三者关系

设计图纸 (算法)



建筑材料 (数据结构)



房子 (程序)





如何描述算法

- 算法的描述方法
 - 自然语言
 - 流程图
 - 程序设计语言：C、C++、Java、Python、...
 - **伪代码(Pseudocode)**——算法语言
 - 类C/类Pascal语言
 - 结构化编程语言



如何描述算法

Algorithm 1 插入排序算法 Insert

Input: 待排序数组 T .

Output: 排序完成数组 T .

类C语言

```
template<class Type>
void Insert(Type T[], int n){
    //从第2列到第n个元素
    //把该元素插入到之前的数组相应位置上
    for(int i=2; i<n; i++){
        Type x = T[i]; int j = i-1;
        while(j > 0 && x < T[j]){
            T[j+1] = T[j]; j = j-1;
        }
        T[j+1] = x;
    }
}
```



如何描述算法

Algorithm 2 插入排序算法 Insert

Input: 待排序数组 T .

Output: 排序完成数组 T .

类Pascal语言

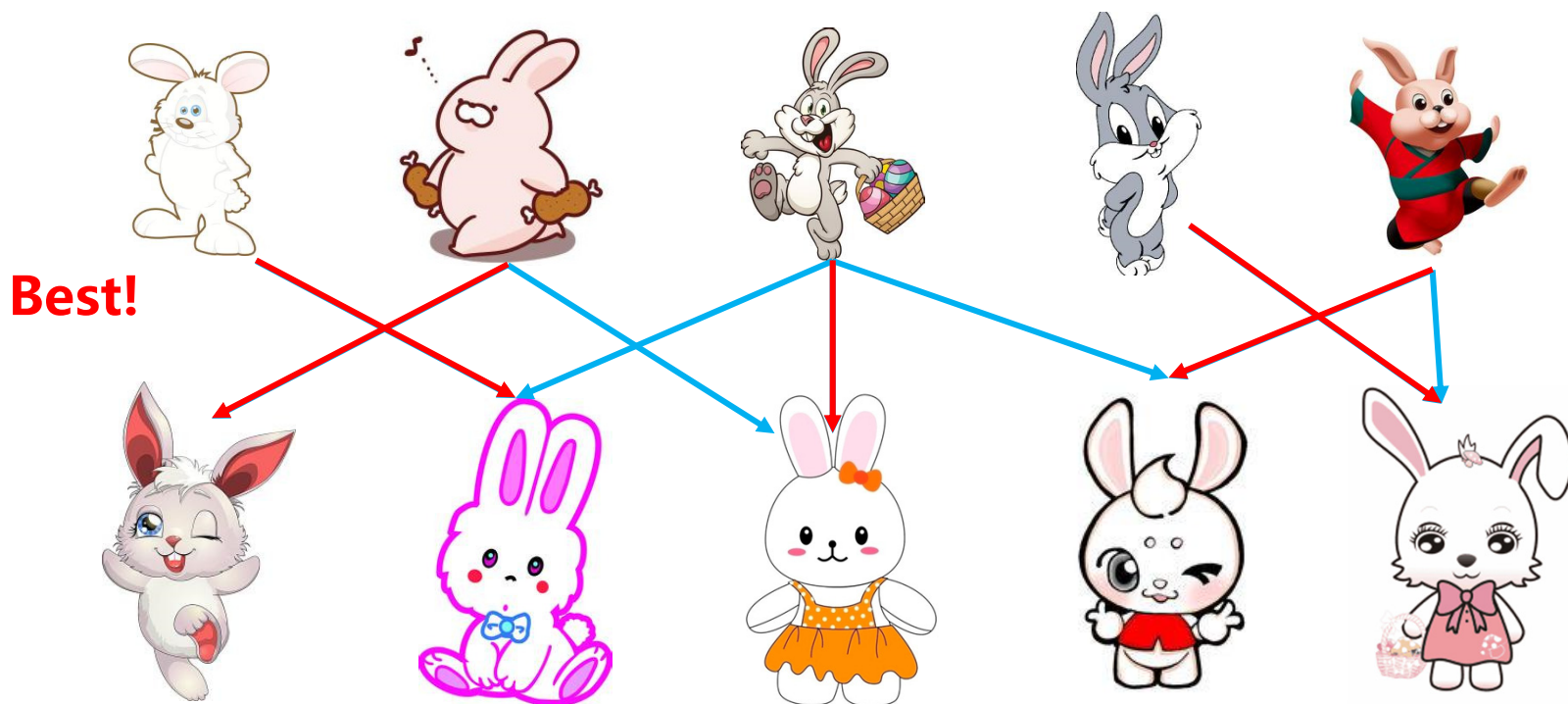
```
Procedure Insert( $T[1...n]$ )  
  //从第 2 列到第  $n$  个元素,  
  //把该元素插入到之前的数组相应位置上  
  for  $i \leftarrow 2$  to  $n$  do  
     $x \leftarrow T[i]; j \leftarrow i - 1;$   
    while  $j > 0$  and  $x < T[j]$  do  
       $T[j + 1] \leftarrow T[j]; j \leftarrow i - 1;$   
    end while  
     $T[j + 1] \leftarrow x;$   
  end for
```

算法设计

如何设计一个算法（例1）

草原上生活着一群可爱的兔子，有5只公兔子和5只母兔子，每只公兔子都对若干母兔子有好感。

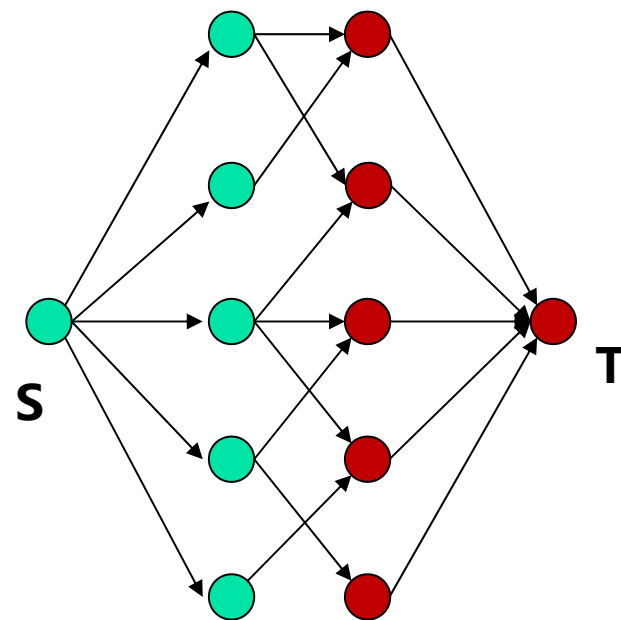
如何帮助尽可能多的公兔子找到伴侣？



如何设计一个算法 (例1)

- 步骤一：抽象为数学问题
 - 将公兔子和母兔子看作图中的点
 - 若公兔子a喜欢母兔子b，则在a和b之间连一条边
 - 增加一个源点S和终点T

最后，兔子匹配问题抽象成为
最大流问题 (第8章)



- 步骤二：将求解过程用计算机语言描述
(较为复杂，此处省略，见第8章)

如何设计一个算法（例2）

假设草原上出现了一对新生的兔子，从第二个月开始每个月都能繁殖出一对新的兔子，而新生的兔子也会从第二个月开始繁殖。如果兔子永不死亡，**请问一年后草原上有几对兔子？**

	1月	2月	3月	4月	5月
刚出生	1	0	1	1	2
一个月	0	1	0	1	1
成熟的	0	0	1	1	2
总 数	1	1	2	3	5



如何设计一个算法（例2）

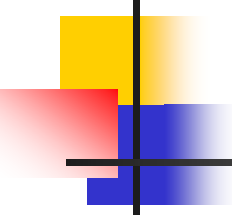
- 兔子对数序列：0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...
- 步骤一：抽象为数学问题 **Fibonacci数列**

- 序列的表达式
$$\begin{cases} f_0 = 0; & f_1 = 1 \\ f_n = f_{n-1} + f_{n-2}, & n \geq 2 \end{cases}$$

- 步骤二：将求解过程用计算机语言描述

```
int Fibonacci(int n){  
    if(n<2) then return n  
    else return Fibonacci(n-1) + Fibonacci(n-2)  
}
```

算法分析

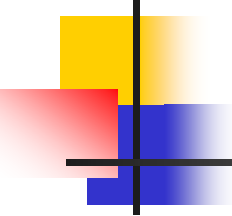


如何选择算法

- 当解决一个问题时，存在几种算法可供选择，如何决定哪个最好？

以排序算法为例：

算法	最坏情况	平均情况
插入排序	$O(n^2)$	$O(n^2)$
冒泡排序	$O(n^2)$	$O(n^2)$
快速排序	$O(n^2)$	$O(n\log n)$
归并排序	$O(n\log n)$	$O(n\log n)$



插入排序算法的插入操作

输入

5	7	1	3	6	2	4
---	---	---	---	---	---	---

前面已经排好序，插入2

插入2

1	3	5	6	7	2	4
---	---	---	---	---	---	---

插入后

1	2	3	5	6	7	4
---	---	---	---	---	---	---

插入排序算法的运行实例

输入

5	7	1	3	6	2	4
---	---	---	---	---	---	---

初始

5	7	1	3	6	2	4
---	---	---	---	---	---	---

插入7

5	7	1	3	6	2	4
---	---	---	---	---	---	---

插入1

1	5	7	3	6	2	4
---	---	---	---	---	---	---

插入3

1	3	5	7	6	2	4
---	---	---	---	---	---	---

插入6

1	3	5	6	7	2	4
---	---	---	---	---	---	---

插入2

1	2	3	5	6	7	4
---	---	---	---	---	---	---

插入4

1	2	3	4	5	6	7
---	---	---	---	---	---	---

冒泡排序算法的一次巡回

输入

5	7	1	3	6	2	4
---	---	---	---	---	---	---

巡回

5	7	1	3	6	2	4
---	---	---	---	---	---	---



巡回后

5	1	3	6	2	4	7
---	---	---	---	---	---	---

冒泡排序算法的运行实例

输入	5	7	1	3	6	2	4
巡回1	5	1	3	6	2	4	7
巡回2	1	3	5	2	4	6	7
巡回3	1	3	2	4	5	6	7
巡回4	1	3	2	4	5	6	7
巡回5	1	2	3	4	5	6	7
巡回6	1	2	3	4	5	6	7

问题的计算复杂度分析

■ 问题

- 排序问题计算难度如何?
- 问题计算复杂度的估计方法?
- 哪个排序算法效率最高?
- 是否可以找到更好的排序算法?



哪个排序算法效率最高?
如何分析排序问题计算复杂度?

$O(n^2)$

插入排序
冒泡排序
选择排序
快速排序

$O(n \log n)$

堆排序
归并排序

?

更好的
排序算法

评估算法的执行效率

■ 两种评估方法：

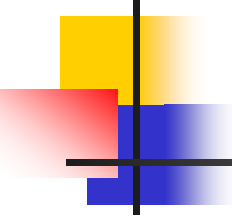
- **经验(Empirical)**：对各种算法编程，用**不同实例**进行实验；

- **理论(Theoretical)**：以数学化的方式确定算法所需要**资源数与实例大小**之间函数关系。

时间/空间

某些方法实例中某些构件数的数量

- 排序：以参与排序的项数表示实例大小；
- 图论：常用图的节点/边来表示



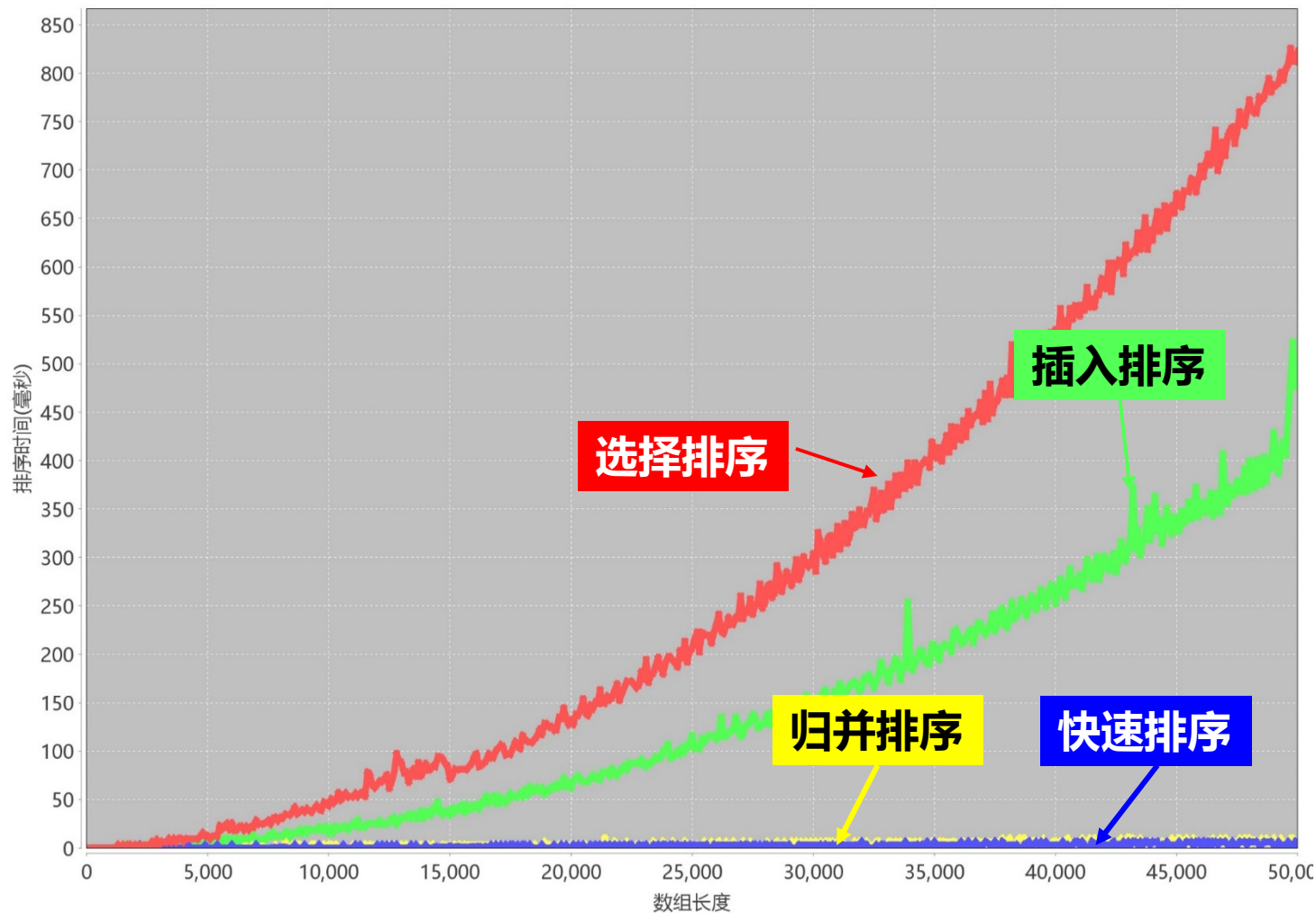
基于经验的评估方法

■ 实验方案

- 生成 n 个随机数并进行排序
($n=100, 200, \dots, 50000$)
- 记录各个算法的排序时间
- 用图的形式画出来

基于经验的评估方法

排序算法执行时间比较图



经验法存在的问题

■ 经验法的问题

- 依赖于计算机
- 依赖于语言/编程技能
- 需要一定的编程/调试时间
- 只能评估部分实例的效率



理论法优点：既不依赖于计算机，也不依赖于语言/编程技能。节省了无谓的编程时间；可研究在任何实例上算法效率。

基于理论的评估方法

■ 执行时间的估计

- 定义评估函数 $T(n)$, n 为输入数据规模
- 如果存在一个正的常数 c , 而该算法对每个大小为 n 的实例的执行时间都不超过 $cT(n)$ 秒
- →该算法的开销在 $T(n)$ 级内。

为什么定义常数 c ?



Apple I
CPU MOS 6502
@ 1 MHz



iMac Pro
CPU Intel Xeon W
@ 3.2GHz 八核



平均和最坏情况分析

- 一个算法的时间耗费，对于不同的实例都会有所不同。可以定义两种时间复杂度：

- 最坏情况下的时间复杂度 $W(n)$

算法求解输入规模为 n 的实例所需要的**最长**时间

- 平均情况下的时间复杂度 $A(n)$

在给定同样规模为 n 的输入实例的概率分布下，算法求解这些实例所需要的**平均**时间



平均和最坏情况分析

■ $A(n)$ 的计算公式

- 设 S 是规模为 n 的实例集
- 实例 $I \in S$ 的概率是 P_I
- 算法对实例 I 执行的基本运算次数是 t_I

$$A(n) = \sum_{I \in S} P_I \times t_I$$

在某些情况下可以假定每个输入的实例概率相等



例子：检索/查找

检索问题

- 输入：
 - 非降顺序排列的无重复数值 L ，元素数 n ，数 x
- 输出： j
 - 若 x 在 L 中， j 是 x 首次出现的下标；
 - 否则 $j=0$
- 基本运算： x 与 L 中元素的比较

顺序检索算法

- $j=1$, 将 x 与 $L[j]$ 比较,
 - 如果 $x=L[j]$, 则算法停止, 输出 j ;
 - 如果不等, 则把 j 加1, 继续比较, 如果 $j>n$, 则停止并输出0.

实例:

1	2	3	4	5
---	---	---	---	---

$x=4$, 需要比较4次

$x=2.5$, 需要比较5次

最坏情况?
平均情况?



最坏情况的时间估计

- 最坏的输入：
 - $x=L[n]$ 或者不在 L 中，要做 n 次比较
- 最坏情况下时间： $W(n)=n$



平均情况的时间估计

- 输入实例的概率分布：
 - 假设 x 在 L 中的概率是 p ，且每个位置概率相等

$$A(n) = \sum_{i=1}^n i \times \frac{p}{n} + (1-p) \times n = \frac{p(n+1)}{2} + (1-p)n$$

当 $p=1/2$ 时

$$A(n) = \frac{n+1}{4} + \frac{n}{2} \approx \frac{3n}{4}$$



思考

- 如何改进之前的顺序检索算法？
- 并分析其最坏情况和平均情况的时间复杂度。

顺序检索算法改进：

- $j=1$ ，将 x 与 $L[j]$ 比较，
 - 如果 $x=L[j]$ ，则算法停止，输出 j ；
 - 如果 $x > L[j]$ ，则把 j 加1，继续比较；如果 $x < L[j]$ ，则停机并输出0；
 - 如果 $j > n$ ，则停机并输出0.